

Guía para defender el código — Breakthrough

Documento de estudio para la interrogación. Explica qué hace cada parte de `main.py`, por qué está escrita así, dónde vive cada regla del anexo, y qué responder ante las preguntas más probables.

1. El programa en 30 segundos

El juego completo son **172 líneas en un solo archivo**, organizadas en siete funciones. La idea central es simple:

El tablero es una **lista de listas de caracteres**. Todo el resto del programa son funciones que leen o modifican esa lista.

El flujo es un ciclo: mientras nadie haya ganado, se dibuja el tablero, se pide una jugada al jugador de turno, se aplica, y el turno pasa al rival.

Función	Responsabilidad en una frase
<code>crear_tablero(n)</code>	Arma el tablero inicial de <code>n × n</code> .
<code>rival(jugador)</code>	Devuelve "B" si recibe "A" y viceversa.
<code>movimientos_de(...)</code>	Destinos legales de una ficha. Aquí viven las reglas de movimiento.
<code>movimientos(...)</code>	Todos los movimientos legales de un jugador .
<code>mover(...)</code>	Aplica una jugada ya validada.
<code>ganador(...)</code>	Dice si la partida terminó y quién ganó.
<code>mostrar(...)</code>	Dibuja el tablero en la terminal.
<code>leer_tamano() / pedir_jugada(...)</code>	Leen datos del usuario y los validan.
<code>main()</code>	El ciclo de turnos.

Si te preguntan "**¿dónde está la lógica del juego?**", la respuesta es: "*en `movimientos_de` y en `ganador`; las demás funciones son entrada, salida o coordinación*".

2. Recorrido del código

2.1 Las constantes

VACIA = "."

MINIMO = 6 # tamaño mínimo de tablero para este juego

AVANCE = {"A": -1, "B": 1} # hacia dónde avanza cada jugador

AVANCE es la pieza clave del diseño. En vez de escribir dos versiones de la lógica —una para A y otra para B— se guarda **cuánto cambia el número de fila** cuando cada jugador avanza. A va hacia la fila 1, así que su fila **disminuye** (**-1**); B va hacia la fila n, así que **aumenta** (**+1**).

Pregunta probable: ¿por qué A es **-1**? Porque la fila 1 está arriba y la fila n abajo. A avanza hacia arriba, y "hacia arriba" significa restar 1 al número de fila. Con este diccionario, la misma función sirve para los dos jugadores.

2.2 crear_tablero(n)

```
tablero = [[VACIA] * n for _ in range(n)]
```

```
for fila in (1, 2):            # B ocupa completas las dos primeras filas
```

```
    tablero[fila - 1] = ["B"] * n
```

```
for fila in (n - 1, n):        # A ocupa completas las dos últimas
```

```
    tablero[fila - 1] = ["A"] * n
```

```
return tablero
```

Primero crea un tablero vacío y después reemplaza cuatro filas completas. Las posiciones se **calculan a partir de n**, por eso funciona con cualquier tamaño sin cambiar nada.

Pregunta probable: ¿por qué `[[VACIA] * n for _ in range(n)]` y no `[[VACIA] * n] * n`? Porque la segunda forma crea **una sola lista repetida n veces**: las n filas serían el mismo objeto en memoria, y al modificar una casilla se modificarían todas las filas a la vez. Es un error clásico de Python. La comprensión de lista crea una lista nueva e independiente en cada vuelta.

Pregunta probable: ¿por qué *fila - 1*? Porque las coordenadas del juego parten en 1 (como en el anexo) pero las listas de Python parten en 0. Se resta 1 al momento de tocar la lista, y así el resto del programa razona siempre con la numeración del anexo.

2.3 *movimientos_de(tablero, jugador, fila, columna)* — el corazón

```
n = len(tablero)
```

```
if not (1 <= fila <= n and 1 <= columna <= n):
```

```
    return []
```

```
if tablero[fila - 1][columna - 1] != jugador:
```

```
    return []
```

```
destino_fila = fila + AVANCE[jugador]
```

```
if not 1 <= destino_fila <= n:
```

```
    return []
```

```
destinos = []
```

```
for lado in (-1, 0, 1):    # diagonal izquierda, recto, diagonal derecha
```

```
    destino_columna = columna + lado
```

```
    if not 1 <= destino_columna <= n:
```

```
        continue
```

```
    contenido = tablero[destino_fila - 1][destino_columna - 1]
```

```
    if contenido == VACIA or (lado != 0 and contenido == rival(jugador)):
```

```
        destinos.append((destino_fila, destino_columna))
```

```
return destinos
```

Se lee así:

1. **Dos guardas iniciales.** Si la coordenada no existe o la ficha no es del jugador, no hay movimientos. Devolver una lista vacía evita que el resto del programa tenga que preguntar "¿es válida esta casilla?" en cada uso.
2. **Una sola fila de destino.** Como el movimiento es siempre de una casilla hacia adelante, `destino_fila` queda fijo: `fila + AVANCE[jugador]`. Eso ya impide moverse hacia atrás o hacia los lados.
3. **Tres columnas candidatas.** `lado` vale `-1`, `0` y `1`: diagonal izquierda, recto y diagonal derecha.
4. **La condición que encierra las reglas.**

La línea más importante del programa es esta:

```
if contenido == VACIA or (lado != 0 and contenido == rival(jugador)):
```

Traducida a palabras:

- `contenido == VACIA` → **a una casilla vacía se puede llegar de las tres formas** (recta o diagonal).
- `lado != 0 and contenido == rival(jugador)` → **solo se captura en diagonal**. El `lado != 0` es justamente lo que impide capturar de frente: cuando `lado == 0` el avance es recto y esta mitad de la condición es falsa.
- Si la casilla tiene una ficha **propia**, ninguna de las dos mitades se cumple, así que el destino no se agrega. Eso implementa "no se puede aterrizar sobre una ficha propia".

Pregunta probable: ¿dónde impides que una ficha capture avanzando recto? → En el `lado != 0` de esa condición.

Pregunta probable: ¿dónde controlas que no se salte sobre otras piezas? No hace falta controlarlo: el movimiento es de una sola casilla, así que nunca hay una casilla intermedia que saltar.

Pregunta probable: ¿capturar es obligatorio? No, y el código lo refleja: cuando hay una captura disponible, el destino vacío recto también se agrega a la lista. El jugador elige.

2.4 `movimientos(tablero, jugador)`

```
return [
```

```
((fila, columna), destino)
```

```

for fila in range(1, n + 1)

for columna in range(1, n + 1)

for destino in movimientos_de(tablero, jugador, fila, columna)

]

```

Recorre el tablero completo y junta los movimientos de todas las fichas del jugador. Devuelve pares (**origen**, **destino**), por ejemplo ((5, 1), (4, 1)).

Fíjate que **no filtra por dueño de la ficha**: no hace falta, porque **movimientos_de** ya devuelve lista vacía si la casilla no es del jugador. Eso evita repetir la misma comprobación en dos lugares.

Pregunta probable: *¿por qué los tres **for** en una sola comprensión?* Es un triple ciclo anidado escrito de forma compacta: filas, columnas y destinos de cada ficha. Equivale a tres **for** con **append**.

Para qué sirve esta función. Se usa en dos partes: para validar la jugada que escribe el usuario (**jugada in legales**) y para detectar si un jugador se quedó sin movimientos. En la Entrega 2 será la **generación de sucesores** que necesita Minimax.

2.5 **mover(tablero, jugador, origen, destino)**

```
tablero[destino[0] - 1][destino[1] - 1] = jugador
```

```
tablero[origen[0] - 1][origen[1] - 1] = VACIA
```

Dos líneas: se escribe la ficha en el destino y se vacía el origen.

Pregunta probable: *¿dónde eliminas la ficha capturada?* No hay que eliminarla explícitamente: al escribir la ficha propia en la casilla de destino, la del rival queda **sobrescrita**. Esa es la captura.

Pregunta probable: *¿por qué no devuelve el tablero?* Porque las listas en Python son **mutables** y se pasan por referencia: la función modifica el mismo tablero que recibió. Devolverlo sería redundante.

2.6 **ganador(tablero, turno)**

```
if "A" in tablero[0]:
```

```
return "A"
```

```
if "B" in tablero[-1]:
```

```
    return "B"
```

```
for jugador in ("A", "B"):
```

```
    if not any(rival(jugador) in fila for fila in tablero):
```

```
        return jugador
```

```
if not movimientos(tablero, turno):
```

```
    return rival(turno)
```

```
return None
```

Las tres condiciones de término del anexo, en orden:

1. **Victoria por avance.** `tablero[0]` es la fila 1 y `tablero[-1]` es la última. Si hay una "A" en la fila 1, A llegó a la fila inicial de B.
2. **Victoria por captura total.** Si ninguna fila contiene fichas del rival, el jugador ganó.
3. **Sin movimientos legales.** El jugador al que le toca mover y no puede, pierde. Por eso la función necesita saber **de quién es el turno**.

Pregunta probable: ¿por qué *ganador* recibe el turno? Porque la tercera condición no depende solo de las fichas en el tablero, sino de **quién debe mover**. La misma posición puede ser terminal para uno y no para el otro.

Pregunta probable: ¿qué significa `tablero[-1]`? En Python el índice `-1` es el último elemento, así que es la fila n sin tener que calcular `len(tablero) - 1`.

Pregunta probable: ¿y los empates? No existen bajo estas reglas, y el anexo lo dice explícitamente. La función devuelve `None` mientras la partida siga y una letra cuando termina; nunca hay un tercer resultado.

2.7 `mostrar`, `leer_tamano` y `pedir_jugada`

Son la interfaz: no contienen reglas, solo entrada y salida.

- `mostrar` imprime los números de fila y columna, y usa `rjust` para que las columnas queden alineadas también con tableros de dos dígitos ($n \geq 10$).

- `leer_tamano` acepta el tamaño como argumento (`python main.py 8`) o lo pregunta, y **rechaza cualquier valor menor que 6**, que es la restricción del enunciado para Breakthrough.
- `pedir_jugada` usa `re.findall(r"\d+", entrada)` para extraer **todos los números** del texto escrito. Así `5 1 4 1`, `5,1 4,1` y `(5,1)-(4,1)` funcionan igual, sin escribir un analizador de texto complicado.
 - Con **cuatro** números arma la jugada y comprueba `jugada in legales`.
 - Con **dos** números muestra los destinos de esa ficha (ayuda para jugar).

Pregunta probable: *¿cómo validas que la jugada sea legal?* No la valido caso por caso: genero la lista de **todos** los movimientos legales y compruebo que la jugada escrita esté en esa lista. Una sola fuente de verdad, imposible que se contradiga con la generación de movimientos.

2.8 `main()`

```
tablero = crear_tablero(leer_tamano())
```

```
turno = "A"          # según el anexo, A realiza la primera jugada
```

```
while ganador(tablero, turno) is None:
```

```
    mostrar(tablero)
```

```
    jugada = pedir_jugada(tablero, turno)
```

```
    if jugada is None:
```

```
        print("Partida interrumpida.")
```

```
        return
```

```
    mover(tablero, turno, *jugada)
```

```
    turno = rival(turno)
```

Pregunta probable: *¿qué hace el asterisco en `mover(tablero, turno, *jugada)`? `jugada` es la tupla (`origen, destino`). El `*` la **desempaqueta** en dos argumentos, equivalente a `mover(tablero, turno, jugada[0], jugada[1])`.*

Pregunta probable: *¿por qué la condición del `while` es una llamada a función y no una variable?* Porque así el estado terminal se evalúa **al comienzo de cada**

turno, que es exactamente lo que pide el anexo para la regla de quedarse sin movimientos.

3. Cada regla del PDF y dónde está en el código

Regla del anexo	Dónde se cumple
B en las filas 1 y 2; A en las filas $n-1$ y n	<code>crear_tablero</code>
A realiza la primera jugada	<code>turno = "A"</code> en <code>main</code>
A avanza hacia la fila 1; B hacia la fila n	diccionario <code>AVANCE</code>
Se mueve una sola ficha, una casilla hacia adelante	<code>destino_fila</code> fijo en <code>movimientos_de</code>
No hay movimientos laterales ni hacia atrás	la fila de destino es la única posible
Avance recto solo a casilla vacía	<code>contenido == VACIA</code> con <code>lado == 0</code>
Avance diagonal a casilla vacía o con captura	segunda mitad de la condición
No aterrizar sobre ficha propia	ninguna mitad de la condición se cumple
No se salta sobre piezas	el movimiento es de una casilla
No hay capturas múltiples	<code>mover</code> ejecuta un único movimiento
Capturar no es obligatorio	los destinos vacíos también se ofrecen
Victoria por llegar a la fila inicial del rival	primeras dos líneas de <code>ganador</code>
Victoria por capturar todas las fichas	ciclo <code>for jugador in ("A", "B")</code>
Pierde quien no tiene movimientos legales	<code>if not movimientos(tablero, turno)</code>
No se permite pasar	no existe ninguna opción de pasar turno
No hay empates	<code>ganador</code> solo devuelve "A", "B" o <code>None</code>
Tablero parametrizable con $n \geq 6$	<code>leer_tamano</code> y la constante <code>MINIMO</code>

Regla del anexo	Dónde se cumple
No hay promoción: la partida termina de inmediato	el <code>while</code> corta apenas hay ganador

4. Preguntas difíciles y cómo responderlas

¿Por qué usaste una lista de listas y no otra estructura? Es la representación más directa de una grilla: `tablero[filas][columnas]`. Un diccionario de coordenadas ocuparía menos memoria en tableros muy vacíos, pero aquí el tablero está bastante lleno y la lista permite imprimirlo y recorrerlo de forma inmediata.

¿Por qué las coordenadas parten en 1 si Python parte en 0? Para hablar el mismo idioma que el anexo y que el jugador. La conversión se hace únicamente al indexar (`fila - 1`), de modo que hay un solo punto de traducción y no se mezclan dos numeraciones a lo largo del código.

¿Puede un jugador quedarse sin movimientos legales de verdad? Está implementado, pero en la práctica es **inalcanzable**. Piensa en la ficha más avanzada de un jugador: para estar bloqueada, sus tres casillas de avance tendrían que estar ocupadas por fichas propias, que estarían **más adelante** que ella —contradicción—, o quedar fuera del tablero, lo que significaría que ya llegó a la fila objetivo y por lo tanto ya ganó. Lo dejé implementado porque el anexo lo exige, y comprobé el razonamiento jugando partidas aleatorias completas: nunca se alcanza esa situación.

(Esta respuesta impresiona: demuestra que entendiste las reglas, no que solo las copiaste.)

¿Qué pasa si escribo una coordenada fuera del tablero? No se cae: `movimientos_de` comprueba los límites y devuelve lista vacía, y `pedir_jugada` responde "movimiento ilegal" y vuelve a preguntar.

¿Cuál es la complejidad de generar los movimientos? `movimientos` recorre las n^2 casillas y para cada ficha revisa 3 destinos, o sea $O(n^2)$. Con $n = 8$ son 64 casillas: instantáneo. Importa para la Entrega 2, porque Minimax llamará a esta función en cada nodo del árbol.

¿Cuántos movimientos legales hay en la posición inicial? $3n - 2$. Las n fichas de la fila $n-1$ pueden avanzar recto y en diagonal, pero las dos de los extremos pierden una diagonal cada una. Con $n = 8$ son 22.

¿Cómo probaste que funciona? Reproduce la **partida simulada de 6 × 6 del anexo**, jugada por jugada, y el tablero final coincide casilla por casilla con el del PDF: A gana en el turno 7 tras capturar en (1, 1). Además verifiqué la configuración inicial para varios tamaños, que el avance recto no capture, que no existan movimientos laterales ni hacia atrás, las tres condiciones de victoria, y partidas aleatorias completas para confirmar que siempre hay un ganador.

¿Cómo agregarías el agente de la Entrega 2? Las dos piezas que Minimax necesita ya existen: `movimientos(tablero, jugador)` genera los sucesores y `ganador(tablero, turno)` detecta los estados terminales. Faltaría una función de evaluación heurística (por ejemplo, diferencia de material más qué tan avanzada está la ficha más adelantada) y la función recursiva con poda Alfa-Beta. En `main` bastaría con preguntar, cuando le toca al agente, qué movimiento eligió en vez de leerlo por teclado.

5. Cambios en vivo que te pueden pedir

El enunciado dice que el profesor puede pedir **cambiar la lógica durante la interrogación**. Estos son los cambios más probables, ya resueltos. Practícalos una vez antes de la presentación.

Que cada jugador empiece con 3 filas de fichas En `crear_tablero`, cambia las dos tuplas:

```
for fila in (1, 2, 3):
```

```
    tablero[fila - 1] = ["B"] * n
```

```
for fila in (n - 2, n - 1, n):
```

```
    tablero[fila - 1] = ["A"] * n
```

(Funciona incluso con $n = 6$, aunque ahí el tablero queda lleno y la partida empieza directamente con capturas.)

Que B juegue primero En `main`, cambia `turno = "A"` por `turno = "B"`.

Que capturar sea obligatorio En `movimientos`, guarda la comprensión en una variable y filtra antes de devolverla. La función completa queda así:

```
def movimientos(tablero, jugador):
```

```
    """Devuelve todos los movimientos legales del jugador en turno."""
```

```

n = len(tablero)

todos = [

    ((fila, columna), destino)

    for fila in range(1, n + 1)

    for columna in range(1, n + 1)

    for destino in movimientos_de(tablero, jugador, fila, columna)

]

capturas = [(origen, destino) for origen, destino in todos

             if tablero[destino[0] - 1][destino[1] - 1] != VACIA]

return capturas or todos

```

El truco está en `capturas or todos`: si hay capturas devuelve solo esas, y si la lista está vacía —que en Python es un valor falso— devuelve todos los movimientos.

Que se pueda capturar también de frente Quita la condición `lado != 0`:

```
if contenido == VACIA or contenido == rival(jugador):
```

Que se permita mover hacia atrás Este es el único cambio que exige reescribir la función completa, porque hay que meter el ciclo de `lado` dentro de un ciclo nuevo de direcciones. Reemplaza `movimientos_de` por esta versión:

```

def movimientos_de(tablero, jugador, fila, columna):

    """Devuelve los destinos legales de la ficha ubicada en (fila, columna)."""

    n = len(tablero)

    if not (1 <= fila <= n and 1 <= columna <= n):

        return []

    if tablero[fila - 1][columna - 1] != jugador:

```

```

    return []

destinos = []

for direccion in (AVANCE[jugador], -AVANCE[jugador]):

    destino_fila = fila + direccion

    if not 1 <= destino_fila <= n:

        continue

    for lado in (-1, 0, 1):

        destino_columna = columna + lado

        if not 1 <= destino_columna <= n:

            continue

        contenido = tablero[destino_fila - 1][destino_columna - 1]

        if contenido == VACIA or (lado != 0

                                and contenido == rival(jugador)):

            destinos.append((destino_fila, destino_columna))

return destinos

```

Que gane quien capture más fichas en 20 turnos Lleva un contador de turnos en `main` y, al llegar al límite, compara `sum(fila.count("A") for fila in tablero)` con lo mismo para "B".

Que el tablero pueda ser rectangular ($n \times m$) Reemplaza `n = len(tablero)` por dos variables: `filas = len(tablero)` y `columnas = len(tablero[0])`, y usa cada una en su comprobación de límites.

6. Traza de una jugada, para explicar el flujo

Si te piden "muéstrame qué pasa cuando muevo", sigue esta secuencia con el ejemplo 5 1 4 1 en un tablero de 6 × 6:

1. `main` comprueba `ganador(tablero, "A") → None`, la partida sigue.
 2. `mostrar(tablero)` dibuja la grilla.
 3. `pedir_jugada` llama a `movimientos(tablero, "A")` y obtiene los 16 movimientos legales de A.
 4. Lees 5 1 4 1; `re.findall` extrae `[5, 1, 4, 1]`; se arma `((5, 1), (4, 1))`.
 5. Esa tupla está en la lista de legales → se devuelve.
 6. `mover` escribe "A" en la fila 4 columna 1 y deja "." en la fila 5 columna 1.
 7. `turno = rival("A") → "B"`.
 8. Vuelve al `while`, que ahora evalúa `ganador(tablero, "B")`.
-

7. Limitaciones que conviene reconocer

Si te preguntan por debilidades, es mejor nombrarlas tú antes de que las encuentre el profesor:

- **No hay opción de deshacer** una jugada. No la pide el enunciado, y para la Entrega 2 se puede agregar guardando la ficha sobrescrita.
- **No se guardan las partidas** en un archivo: todo vive en memoria.
- **La validación de entrada es tolerante**: `re.findall` ignora cualquier texto que no sean números, así que `mover de 5 1 a 4 1` también funciona. Es cómodo para el usuario, pero acepta escrituras poco estrictas.
- **El tablero se dibuja siempre con la fila 1 arriba**, incluso para B, que avanza hacia abajo. Es coherente con el anexo, aunque a B le exige un pequeño esfuerzo mental.